Table of contents

Cross Entropy

Question

Explain cross entropy and give some examples of its use.

Show the link between cross entropy and relative entropy.

Introduction to Cross-Entropy

Cross-entropy is a fundamental measure in information theory and machine learning that quantifies the difference between two probability distributions. It's particularly useful for evaluating the performance of prediction models.

Mathematical Definition

For two discrete distributions $p(x)$ and $q(x)$ defined on the same space $\mathcal{X}$, cross-entropy is defined as:

$$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log_2 q(x) = \mathbb{E}_{p(x)}[-\log_2 q(x)]$$

For continuous distributions with densities $f(x)$ and $g(x)$:

$$H(f, g) = -\int f(x) \log_2 g(x) dx$$

Intuitive Interpretation

Cross-entropy measures **the average number of bits needed** to encode events from distribution p using a code optimized for distribution q.

Postal Code Metaphor

Imagine you need to send letters to different cities: - $p(x)$: Actual sending frequency (Paris 50%, Lyon 30%, Marseille 20%) - $q(x)$: Code you use (optimized for Paris 40%, Lyon 40%, Marseille 20%) - $H(p)$: Optimal code length if you knew the true frequency - $H(p, q)$: Actual length with your imperfect code

```
graph TD
    A[True distribution p(x)] --> B{Code designed for q(x)};
    B --> C[Number of bits used: H(p,q)];
    C --> D[= H(p) + D(p q)];
    D --> E[Optimal bits + penalty];

    F[If q = p] --> G[H(p,q) = H(p)];
    F --> H[Perfectly adapted code];

    I[If q  p] --> J[H(p,q) > H(p)];
    I --> K[Extra cost due to wrong code];
```

Cross-Entropy Properties

1. Non-Symmetry

$$H(p, q) \neq H(q, p) \quad \text{in general}$$

Unlike a distance, cross-entropy is not symmetric.

2. Lower Bound by Entropy

$$H(p, q) \geq H(p)$$

Equality $H(p, q) = H(p)$ occurs if and only if $p = q$.

3. Convexity with Respect to q

For fixed p, $H(p, q)$ is convex in q, which facilitates optimization.

Fundamental Relationship with Relative Entropy

Mathematical Link

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \parallel q)$$

where: - $H(p)$ is the entropy of distribution p - $D_{\mathrm{KL}}(p \parallel q)$ is the Kullback-Leibler divergence (relative entropy)

Detailed Interpretation

This relationship shows that cross-entropy decomposes into two parts:

1. $H(p)$: The intrinsic uncertainty of the true distribution
    - Theoretical minimum that cannot be compressed
    - Depends only on p, not on our model
2. $D_{\mathrm{KL}}(p \parallel q)$: The penalty due to the wrong model
    - Measures how much q differs from p
    - Reducible by improving our model

```
graph LR
    A[Cross Entropy H(p,q)] --> B[Equivalent to];
    B --> C[H(p) + D(p q)];

    D[Objective: minimize H(p,q)] --> E{Since H(p) is constant};
    E --> F[Minimize D(p q)];
    F --> G[Find q   p];

    H[In practice] --> I[We minimize H(p,q)];
    I --> J[Because p is unknown];
    J --> K[We estimate p from data];
```

Concrete Examples of Use

1. Machine Learning: Classification

Scenario: Image classification with 3 classes (cat, dog, bird)

Training data: - Image 1: true class = “cat” $\rightarrow p = [1, 0, 0]$ - Image 2: true class = “dog” $\rightarrow p = [0, 1, 0]$

Model predictions: - For image 1: $q = [0.7, 0.2, 0.1]$ - For image 2: $q = [0.1, 0.6, 0.3]$

Cross-entropy calculation: For image 1:

$$H(p, q) = -[1 \times \log_2(0.7) + 0 \times \log_2(0.2) + 0 \times \log_2(0.1)] = -\log_2(0.7) \approx 0.514 \text{ bits}$$

For image 2:

$$H(p, q) = -[0 \times \log_2(0.1) + 1 \times \log_2(0.6) + 0 \times \log_2(0.3)] = -\log_2(0.6) \approx 0.737 \text{ bits}$$

Average loss: Average over all images in the training set.

2. Data Compression

Scenario: Compression of English text

Real distribution (p): - 'e' : 12.7% - 't' : 9.1% - 'a' : 8.2% - ... (other letters)

Model used (q): - Uniform model: each letter = 1/26 3.85% - Shakespeare-based model: specific distribution

Efficiency calculation: With uniform model:

$$H(p, q) = -\sum p(x) \log_2(1/26) = \log_2(26) \approx 4.7 \text{ bits/letter}$$

With a good linguistic model (q close to p):

$$H(p, q) \approx H(p) \approx 4.1 \text{ bits/letter}$$

Gain: 0.6 bits per letter, about 18% savings.

3. Language Model Evaluation

Scenario: GPT model evaluated on sentences

Method: Perplexity = $2^{H(p,q)}$

Interpretation: - Low perplexity → model predicts next words well - High perplexity → model is uncertain

```
graph TD
    A[Test text] --> B[Compute H(p,q)];
    B --> C[Perplexity = 2^H(p,q)];
    C --> D{Quality measure};
    D --> E[Low perplexity: good model];
    D --> F[High perplexity: poor model];

    G[Example] --> H[GPT-3 model];
    H --> I[Perplexity ~20-30];
    I --> J[Very performant];

    K[Random model] --> L[Perplexity = vocabulary size];
    L --> M[Very poor];
```

Comparative Table: Entropy vs Cross-Entropy vs KL Divergence

Concept	Formula	Interpretation	Key Property
Entropy $H(p)$	$-\sum p \log p$	Intrinsic uncertainty	Theoretical minimum
Cross-entropy $H(p,q)$	$-\sum p \log q$	Cost of wrong model	Always $\geq H(p)$
KL Divergence $D(p\|q)$	$\sum p \log(p/q)$	Wrong model penalty	Difference measure

Practical Application: Loss Function in Deep Learning

PyTorch Implementation

```
import torch
import torch.nn as nn

# Data: 3 samples, 5 classes
predictions = torch.tensor([[0.1, 0.2, 0.6, 0.1, 0.0],
                            [0.8, 0.1, 0.05, 0.05, 0.0],
                            [0.05, 0.1, 0.1, 0.15, 0.6]])

true_labels = torch.tensor([2, 0, 4])  # Indices of true classes
```

```
# Cross-entropy calculation
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(predictions, true_labels)
print(f"Cross-entropy loss: {loss.item():.4f}")
```

Step-by-Step Explanation

1. Model produces logits (unnormalized)
2. Softmax converts to probabilities q
3. Comparison with one-hot labels p
4. Calculation of $H(p, q)$
5. Gradient to adjust weights

Visualization of Concept Relationships

```
graph TD
    A[True distribution p] --> B[Entropy H(p)];
    A --> C[Cross-entropy H(p,q)];
    C --> D[KL Divergence D(p q)];

    B --> E[Minimum value];
    D --> F[Deviation from optimal];
    E --> C;
    F --> C;

    G[Learning objective] --> H[Minimize H(p,q)];
    H --> I[Equivalent to minimizing D(p q)];
    I --> J[Since H(p) constant];

    K[Optimal result] --> L[q = p];
    L --> M[H(p,q) = H(p)];
    L --> N[D(p q) = 0];
```

Important Special Cases

1. Binary Case (Binary Classification)

For $p \in \{0, 1\}$ and $q \in [0, 1]$:

$$H(p, q) = -[p \log_2 q + (1 - p) \log_2 (1 - q)]$$

This is the **logistic loss** used in logistic regression.

2. Uniform Distribution as Reference

If q is uniform over N classes:

$$H(p, q) = -\sum p(x) \log_2 (1/N) = \log_2 N$$

All models are compared to this baseline.

Why Minimize Cross-Entropy?

1. Theoretical Justification: Maximum Likelihood Principle

Minimizing $H(p, q)$ is equivalent to maximizing the likelihood of the data.

Proof: Let p_{data} be the empirical distribution, q_θ our model:

$$\min_\theta H(p_{\text{data}}, q_\theta) \equiv \max_\theta \mathbb{E}_{x \sim p_{\text{data}}} [\log q_\theta(x)]$$

2. Desirable Properties

- **Convexity**: Guarantees global minima (for many models)
- **Error Sensitivity**: Strongly penalizes confidence errors
- **Probabilistic Interpretation**: Directly related to probabilities

Detailed Example: MNIST Classification

Problem

Recognize handwritten digits (0-9).

Data

- 28×28 pixel images
- Label: digit from 0 to 9

Model

Neural network with softmax output producing $q = [q_0, q_1, ..., q_9]$.

Loss Calculation

For an image of digit "3" ($p = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$):

- If $q = [0, 0, 0, 0.9, 0, 0, 0, 0, 0.1, 0] \rightarrow H(p, q) = -\log_2(0.9) \approx 0.152$
- If $q = [0.1, 0.1, 0.1, 0.2, 0.1, 0.1, 0.1, 0.1, 0.1, 0] \rightarrow H(p, q) = -\log_2(0.2) \approx 2.322$

The first prediction is much better.

Historical Evolution and Alternatives

1. Mean Squared Error Loss

- Previously popular in regression
- Less suitable for classification
- No direct link to probabilities

2. Advantages of Cross-Entropy

- **Well-behaved Gradients**: Avoids learning plateaus
- **Probabilistic Interpretation**: Outputs are probabilities
- **Information Theory**: Solid foundations

Pitfalls and Best Practices

1. Numerical Stability

Direct calculation $\log(q)$ can be unstable for $q \approx 0$.

Solution: Use log-sum-exp function:

$$H(p, q) = -\sum p(x) \left[z(x) - \log \sum \exp(z(x')) \right]$$

where $z(x)$ are logits.

2. Regularization

Adding a regularization term amounts to:

$$H(p, q) + \lambda R(\theta)$$

This prevents overfitting by penalizing complex models.

3. Imbalanced Data

When some classes are rare:

- **Weighting**: $H_w(p, q) = -\sum w(x)p(x) \log q(x)$
- **Resampling**: Adjust training distribution

Pedagogical Summary

Guessing Game Analogy

Imagine a game where you must guess an animal:

- p: Real animal (e.g., 80% cat, 20% dog)
- q: Your guessing strategy

1. **Entropy** $H(p)$: Optimal number of questions if you knew the true distribution
2. **Cross-entropy** $H(p, q)$: Number of questions with your strategy
3. **KL Divergence** $D(p\|q)$: Extra questions due to your poor strategy

Mnemonic Formula

"Total cost = base uncertainty + ignorance penalty"

$$H(p, q) = H(p) + D(p\|q)$$

Conclusion

Cross-entropy is a powerful measure that:

1. **Evaluates Models**: Measures how well a model q fits data p
2. **Guides Learning**: Optimizable loss function via gradient
3. **Connects Theory and Practice**: Links information theory and ML

The fundamental link with relative entropy shows that minimizing cross-entropy amounts to minimizing the gap between our model and reality, while acknowledging that some uncertainty ($H(p)$) is incompressible.

This unified understanding explains why cross-entropy has become the standard loss function for classification problems and many other tasks in artificial intelligence.